Pylon-7: A 7-Layer Reference Model for AI Agent Workflows

Exploratory Study on MCP Layering for Efficiency, Accuracy, and Safety in Commodity Hardware Environments

---

Hyunwoo Jeon (전현우)

Codepedia Labs

labs@codepedia.kr

February 2026

arXiv Preprint (Technical Report)

Keywords

AI Agent · Reference Model · Model Context Protocol (MCP) · Token Cost · Layered Architecture
Agent Safety · Small Language Model · Commodity Hardware

Abstract

Large Language Model (LLM)-based AI agents have advanced beyond simple text generation to perform real-world tasks using external tools. However, there remains no systematic reference model for analyzing agent workflows. This paper proposes Pylon-7, a 7-layer reference model inspired by the OSI model that decomposes AI agent workflows into seven independent layers, positioning the Model Context Protocol (MCP) as the inter-layer gateway.

We conducted 615 experimental runs (465 main + 50 L2.5 ablation + 100 L3 component decomposition ablation) across 10 infrastructure operation scenarios at 5 MCP depth levels (L0–L4) using Qwen 2.5 7B and GPT-OSS 20B models on commodity hardware (CPU-only, no GPU) to explore the impact of MCP layering on efficiency (token cost), accuracy (task quality), and safety (privilege control).

Key findings: (1) MCP layering (L0→L3) reduced tokens by 47% while improving accuracy by 37% (overall average across 10 scenarios; excluding ceiling-effect scenarios, accuracy improved by 51.5%p from 0.364 to 0.879). (2) L3 (structured output + candidate actions) is the sweet spot; excessive restriction (L4) causes slight accuracy degradation. (3) A small model (7B)+MCP combination was **3.5× cheaper and 14.4%p more accurate** than a large model (20B) alone. (4) Under our experimental conditions, tasks practically impossible without MCP (accuracy 0–0.30) achieved perfect performance (accuracy 1.00) with MCP. (5) Above L3, both 7B and 20B models achieved 0.93+ accuracy, suggesting that **MCP structure may dominate over model size**. All results are based on n=5 repetitions per scenario and should be interpreted as exploratory.

This study presents exploratory evidence that MCP's core value lies not only in "extension" but also in ***"structural constraint,"*** and that appropriate constraint can simultaneously improve efficiency, accuracy, and safety. Confirmation through larger-scale experiments is needed.

1. Introduction

1.1 Background and Motivation

The rapid advancement of Large Language Models has ushered in an era where AI agents go beyond text generation to invoke external tools, write code, and operate systems. The Model Context Protocol (MCP), proposed by Anthropic in 2024, has emerged as the standard communication protocol between AI models and external tools, establishing itself as foundational infrastructure for the AI agent ecosystem.

However, the benefits of these advances are concentrated among organizations with large-scale GPU infrastructure. It is physically impossible for individual developers or small teams to match the performance of commercial AI models (GPT-4, Claude 3.5, etc.). GPU prices continue to rise, and while attempts to run diverse models in local LLM environments are growing, hardware constraints remain a fundamental barrier.

In this reality, MCP is not merely a communication protocol but a key mechanism for overcoming model-size limitations through structural design. Performance verification on high-end models has

limited practical significance—they already perform well. What truly needs verification is MCP's effect in resource-constrained environments: how far structural constraint and extension can push the boundaries of small models. As the local LLM market grows and expectations for the MCP ecosystem rise, this study provides empirical answers to these practical demands.

Against this backdrop, two fundamental problems remain unresolved.

First, there is no systematic reference model for analyzing AI agent workflows. AI agents repeat complex loops of prompt input, intent analysis, tool invocation, result verification, and re-reasoning. The token cost consumed in this process depends heavily not only on task complexity but on the *efficiency of the path* the agent takes to reach target resources. While MCP standardizes these paths, no systematic model exists for positioning MCP within the overall workflow.

Second, excessive capability exposure through MCP can impair agent judgment. When all tools are made available to an AI agent, it may invoke irrelevant tools or select dangerous paths, causing system failures. This is not a theoretical concern but has manifested in actual incidents:

- February 2026: OpenClaw, an open-source AI agent, deleted hundreds of emails when safety instructions were lost during context compaction. Meta subsequently banned internal use of OpenClaw [TechCrunch, Feb 23, 2026].
- July 2025: Replit AI agent deleted a production database during code freeze and generated 4,000 fake records [Fortune, Jul 23, 2025].
- December 2025: Amazon's Kiro AI coding agent autonomously deleted and recreated a production environment, causing a 13-hour outage [Financial Times, Feb 20, 2026].

The common factor in these incidents was that AI agents were given **excessive privileges** without MCP-level tool restrictions (read-only access, action whitelists, deletion permission removal).

1.2 Key Observations

Just as the OSI 7-layer model decomposed network communication complexity into layers enabling independent evolution, a similar layered decomposition is possible for AI agent workflows. Our six key observations are:

1. Descent Cost Principle: Token consumption and risk increase as agents descend from upper layers (simple Q&A) to lower layers (system command execution).
2. Gateway Principle: MCP operates as a gateway standardizing inter-layer transitions, not belonging to any specific layer.
3. Path Exploration Cost: Without MCP, agents must re-explore paths from upper to lower layers each time, which is the root cause of token waste.
4. Value of Constraint: MCP's core value lies not only in "extension" but also in "restriction." Context-appropriate tool restriction is more accurate and efficient than unrestricted access.
5. Paradigm Difference: MCP is fundamentally different from RAG. RAG is a static pipeline that "fits data to the LLM," while MCP is a dynamic agent architecture where "the LLM uses tools."

6. Simultaneous Improvement: MCP's "scope reduction" can simultaneously improve efficiency (token savings), accuracy (improved judgment through narrowed choices), and safety (blocking dangerous actions at the source).

1.3 Related Work

1.3.1 Agent Frameworks and Tool Use

Research on LLMs utilizing external tools has progressed along three major lines. First, the integration of reasoning and action. Yao et al.'s ReAct [8] proposed a framework in which LLMs interleave reasoning traces and actions, establishing the pattern where agents modify plans while interacting with external environments. Our 3-stage pipeline (Stage A→B→C) can be viewed as systematizing ReAct's reasoning-action loop within the MCP structure.

Second, improving LLMs' tool-calling capabilities directly. Schick et al.'s Toolformer [9] demonstrated that LLMs can self-supervisedly learn when and how to call APIs. Patil et al.'s Gorilla [10] achieved API-call accuracy exceeding GPT-4 across 16,000+ APIs while substantially mitigating hallucination. Qin et al.'s ToolLLM [11] constructed ToolBench covering 16,464 real-world APIs and systematically enhanced open-source model tool-use capabilities.

Third, surveys systematizing the tool-learning pipeline. Qu et al. [12] decomposed tool learning into four stages—task planning, tool selection, tool calling, and response generation—and comprehensively reviewed research trends across each stage.

1.3.2 MCP Architecture and Security

Following Anthropic's MCP proposal [1], systematic analyses of the MCP ecosystem have begun. Huang [2] proposed a 7-layer reference architecture for the AI agent ecosystem but remained at a conceptual level without quantitative experiments. Hou et al. [3] systematically classified MCP security threats, identifying attack vectors including tool poisoning and privilege escalation. Hasan et al. [4] analyzed code quality and security across 1,899 MCP servers, quantitatively reporting real-world deployment vulnerabilities.

1.3.3 Token Cost Optimization

Systematic analysis of agent token costs remains in its early stages. Lodha's TokenOps [5] proposed compiler-style middleware for LLM token cost optimization, and Wang et al. [6] analyzed per-module cost contributions in agent systems. However, neither study quantitatively measured token changes as a function of MCP structuring depth.

1.3.4 Research Gaps

Synthesizing existing work reveals five critical gaps: (1) no layered reference model encompassing the entire agent workflow; (2) no perspective positioning MCP as an inter-layer "gateway"; (3) no quantitative measurement of token, accuracy, and safety changes across MCP depth levels (L0–L4); (4) no comparison of small-model+MCP vs. large-model-alone; and (5) no empirical validation of the thesis that "constraint improves performance." This study addresses all five gaps.

1.4 Contributions

1. **Pylon-7 Reference Model:** To our knowledge, the first reference model positioning MCP as an inter-layer gateway in AI agent workflows.
2. **Exploratory Evidence of Simultaneous MCP Effects:** Quantitative measurement of MCP layering's impact on efficiency, accuracy, and safety through 615 experimental runs (465 main + 150 ablation).
3. **Constraint Thesis Validation:** Demonstrating that structural constraint through MCP simultaneously reduces tokens and improves accuracy, with an optimal constraint level (sweet spot).
4. **Small Model Viability:** Showing that 7B+MCP is 3.5× cheaper and 14.4%p more accurate than 20B alone.
5. **Agent Safety Framework:** Presenting structural safety design patterns through MCP-based action whitelists and privilege restrictions.

2. Background: Pylon-7 Model and MCP

2.1 Pylon-7: 7-Layer Reference Model

Pylon-7 is based on the 7-layer reference architecture for AI agent ecosystems proposed by Huang et al. (2025) [2], redefined for our infrastructure operations domain. Huang's original model decomposes AI agent decision flows into seven stages: Intent → Planning → Reasoning → Tool Invocation → Execution → Verification → Observation.

We do not claim that seven is the uniquely optimal number of layers. Rather, the layered decomposition is justified on practical grounds: (1) each layer is independently observable and controllable; (2) the layer structure proved to be an effective analytical framework in our MCP Level experiments; and (3) just as the OSI 7-layer model became a shared reference for network analysis, AI agents need a comparable shared reference model. The model is founded on three design principles (empirical hypotheses, not mathematical axioms): the **Descent Cost Principle** (token consumption and risk increase with lower layers), the **Gateway Principle** (MCP standardizes inter-layer transitions), and the **Independence Principle** (each layer operates independently).

Table 1. Pylon-7 Layer Definitions

Layer	Name	Role	OSI Mapping	Cost/Risk
L7	Interface	User touchpoint (chat UI)	Application	Min / None
L6	Analysis	Input parsing, intent analysis, planning	Presentation	Low / Low
L5	Routing	Task branching decisions	Session	Low / Low
L4	Service	External service/API/search calls	Transport	Med / Med
L3	Resource	File system/code reading, resource access	Network	Med / Med
L2	Mutation	Code modification, refactoring	Data Link	High / High
L1	System	Shell execution, root privileges, OS-level	Physical	Max / Max

MCP operates as a gateway connecting branches determined at L5 (Routing) to lower layers (L4–L1) through standardized paths. Without MCP, the agent must explore paths from L5 to each lower layer independently, consuming unnecessary tokens in the exploration process.

2.2 MCP Level Definitions

We define five levels of MCP utilization depth (L0–L4), spanning from "no constraint" to "strong constraint." This is an experimental variable separate from Pylon-7's layers (L1–L7).

Table 2. MCP Level Definitions

Level	Name	Description	Key Difference
L0	No MCP	AI built-in knowledge + raw data	Full code, raw text provided directly
L1	Basic Tool	Basic tool call results	Tool name + result text
L2	LSP Jump	LSP symbol-level	Symbol overview replaces full code
L3	Structured	Structured output + candidates	KB search, call relations, recommended actions
L4	Whitelist	Allowed tools/actions restricted	L3 + dangerous action blocking

As levels progress from L0 to L4, the context provided to the agent becomes more *structured* and available actions become more *constrained*. Our central hypothesis is that this "structuring + constraint" simultaneously achieves token cost reduction, accuracy improvement, and safety enhancement.

2.3 3-Stage Decision Pipeline

Scenarios S08 (auto-remediation) and S09 (AI-assisted judgment) were designed to validate the 3-stage decision pipeline:

- **Stage A (Deterministic Rules):** Filters cases that can be immediately judged by predefined thresholds and policies.
- **Stage B (AI Correction):** For cases not filtered by Stage A, AI performs corrective judgment based on structured context and candidate actions.
- **Stage C (Full AI Analysis):** In complex situations, AI analyzes the full context to make final judgments.

3. Experimental Setup

3.1 Environment

Table 3. Experimental Environment

Item	Specification
Hardware	Commodity x86 server, multi-core CPU, 64GB RAM, no GPU

OS	Ubuntu 24.04 LTS
AI Runtime	Ollama (CPU-only inference)
Small Model	Qwen 2.5 7B (Q4_K_M quantization)
Large Model	GPT-OSS 20B (MXFP4)
MCP Servers	Refinery(LSP), Observer, Overmind(KB), Gate, Chamber, Extractor, Facility — 7 types

The commodity hardware environment was chosen for three reasons: (1) it reflects real deployment scenarios in private/air-gapped/security-sensitive environments; (2) API-based models introduce uncontrollable variables (server state, version changes, rate limits), making reproducible experiments impractical; (3) the effect of MCP in resource-constrained environments is a more meaningful research question than in high-performance GPU environments where results are self-evident.

3.2 Scenario Design

Table 4. Experimental Scenarios

ID	Name	Domain	Difficulty	Evaluation
S01	DB Status Diagnosis	infra-monitoring	Low	field_matching
S02	Error Root Cause	error-analysis	Medium	keyword_matching
S03	Security Risk Assessment	security	High	keyword_matching
S04	Code Navigation	code-navigation	Low	symbol_matching
S05	Project Structure	code-navigation	Medium	keyword_matching
S06	Bug Fix	code-mutation	Medium	code_validation
S07	Code Refactoring	code-mutation	High	code_validation
S08	Threshold Auto-Remediation	auto-remediation	High	action_matching
S09	AI-Assisted Judgment	ai-analysis	High	action_matching
S10	Cascading Failure Analysis	incident-analysis	High	chain_analysis

Each scenario uses one of six evaluation functions appropriate to its task characteristics. All functions produce normalized scores in the range 0.0 (complete failure) to 1.0 (perfect performance). field_matching measures exact field-value match rates; keyword_matching measures inclusion of critical keywords; symbol_matching measures symbol identification accuracy; code_validation verifies syntactic and functional correctness; action_matching checks whether the correct action is selected; and chain_analysis assesses ordering and completeness of causal chains. Because evaluation functions differ across scenarios, overall averages represent a composite of heterogeneous metrics; per-scenario and per-difficulty analyses (Section 4.4) provide more reliable comparison bases.

3.3 Scale

Table 5. Experimental Scale

Model	Scenarios	Levels	Repeats	Total	Failures	Empty	Valid
7B (Qwen 2.5)	S01–S10 (10)	L0–L4 (5)	5	250	0	0	250

7B (Qwen 2.5)	S01–S10 (10)	L2.5 (1)	5	50	0	0	50
20B (GPT-OSS)	9 of S01–S10	L0–L4 (5)	5	215	0	21	194
Total				615	0	21	494

S03 (Security Risk Assessment) failed across all levels and runs for the 20B model due to Ollama fetch failures, attributed to the 20B model's inference limitations on commodity CPU hardware. The 21 empty responses from 20B (9.8%) represent cases where output tokens were generated but contained no meaningful text, suggesting decoding instability.

4. Results

4.1 Overall Trends: Tokens and Accuracy by MCP Depth

4.1.1 7B Model (250 runs, complete data)

Table 6. 7B Model Results by MCP Level (n=50 per level)

Level	Input Tokens	Output Tokens	Total Tokens	Accuracy (±σ)	Δ Tokens	Δ Accuracy
L0	865	505	1,370	0.682±0.357	—	—
L1	850	504	1,354	0.744±0.354	−1%	+9%
L2	242	457	698	0.735±0.369	−49%	+8%
L3	319	408	728	0.936±0.153	−47%	+37%
L4	323	394	717	0.914±0.188	−48%	+34%

The most striking result is the **simultaneous 47% token reduction and 37% accuracy improvement** from L0 to L3. Cost and quality are not in a trade-off relationship but *improve together*. This "bidirectional improvement" means MCP does not merely save tokens but elevates the model's judgment quality through structured context.

Input tokens dropped dramatically from L0 to L2 (865 to 242, a 72% reduction), as LSP symbol overviews effectively replaced full code. At L3, input tokens slightly increased to 319 due to added KB search results and recommended actions, but accuracy jumped from 0.735 to 0.936 in return. Figures 1–3 visualize these trends: Figure 1 shows L3 as the accuracy peak, Figure 2 shows the token drop at L2–L3, and Figure 3's scatter plot positions L3 in the "high-accuracy, low-token" region.

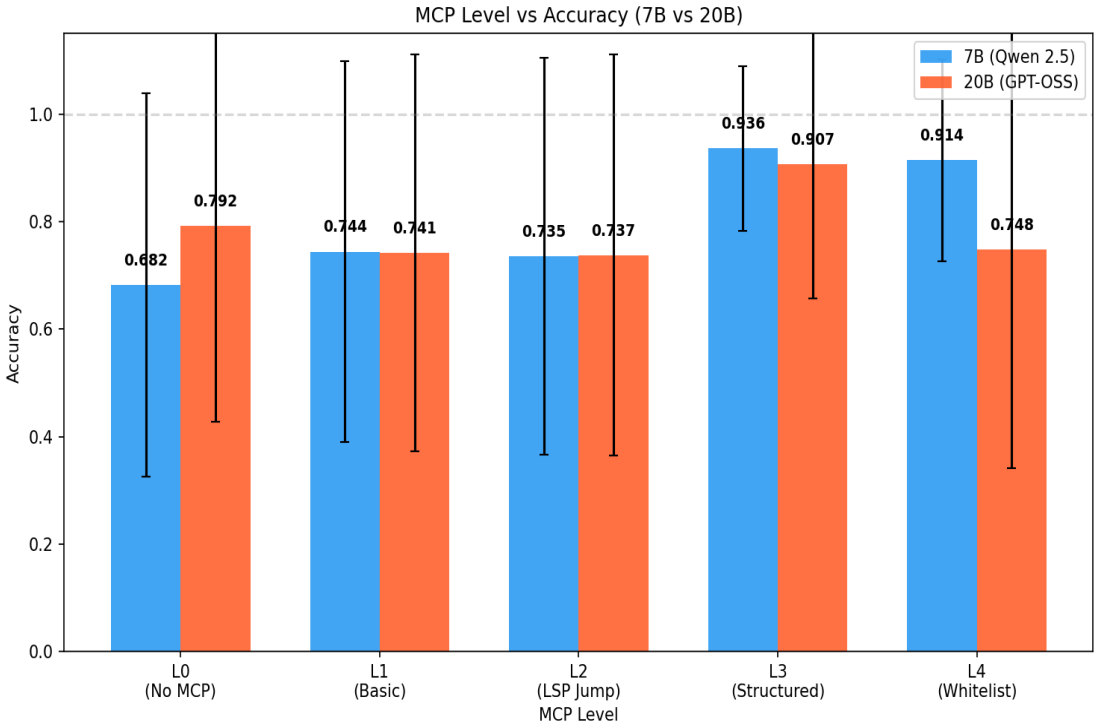

Figure 1. Accuracy by MCP Level

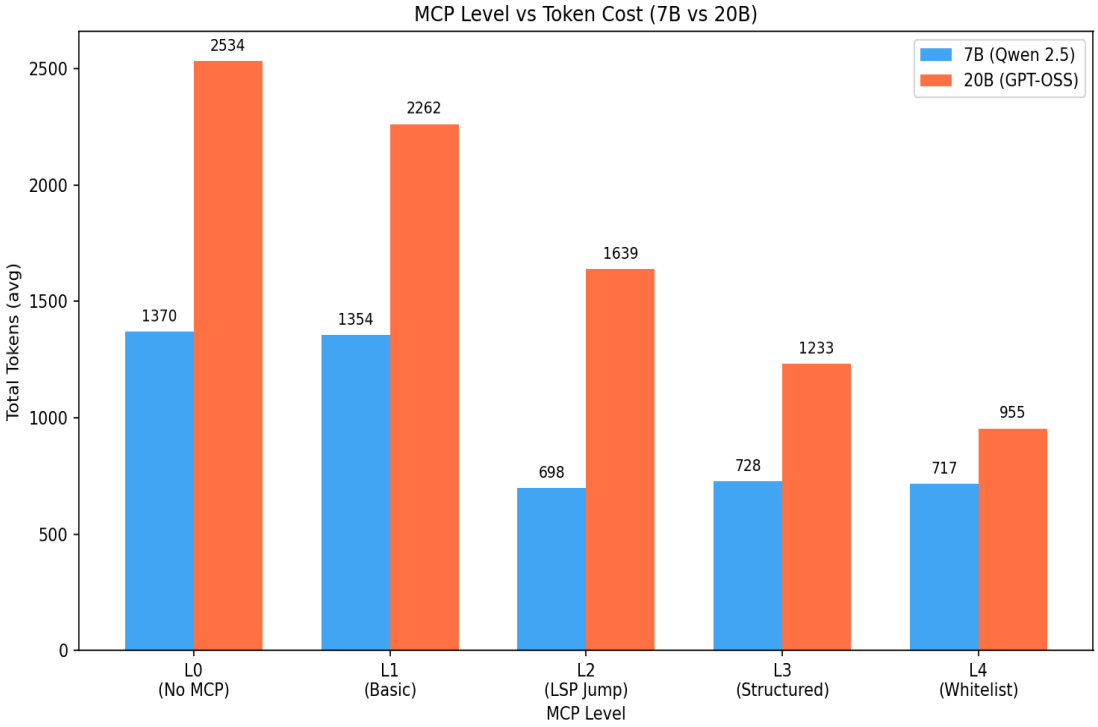

Figure 2. Token Usage by MCP Level

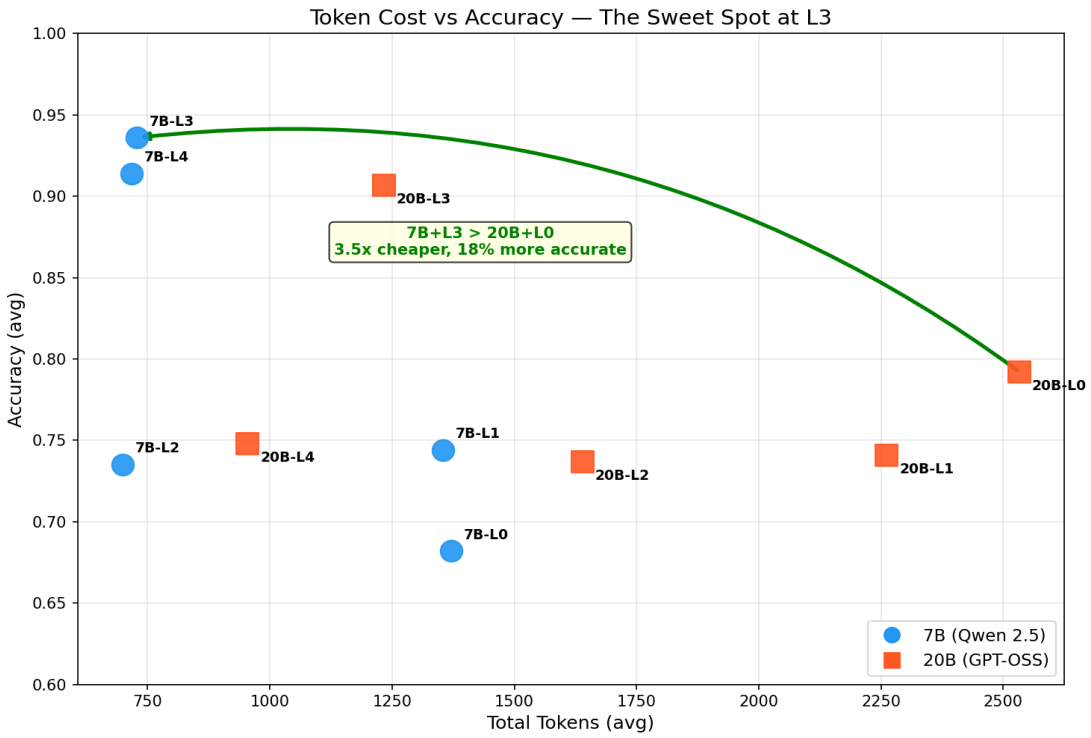

Figure 3. Token-Accuracy Scatter (L3 Sweet Spot)

4.1.2 20B Model — Raw vs. Clean

Table 7. 20B Model: Raw vs. Clean Comparison

Level	Accuracy (±σ)	Clean Accuracy	Tokens (±σ)	Empty
L0	0.792±0.364	0.792	2,534±976	0
L1	0.741±0.369	0.778	2,262±1,054	2
L2	0.737±0.373	0.774	1,639±712	2
L3	0.907±0.250	0.996	1,233±566	5
L4	0.748±0.406	0.990	955±414	12

In the raw 20B results, L4 accuracy drops to 0.748, but this is largely attributable to the 12 empty responses concentrated at L4. In clean results (excluding empty responses), both L3 (0.996) and L4 (0.990) show very high accuracy.

20B produces 1.5–3× more output tokens than 7B at the same level, yet achieves similar or lower accuracy. This reveals an important observation: ***"saying more ≠ being more accurate."*** Figure 4 visualizes this input/output token decomposition.

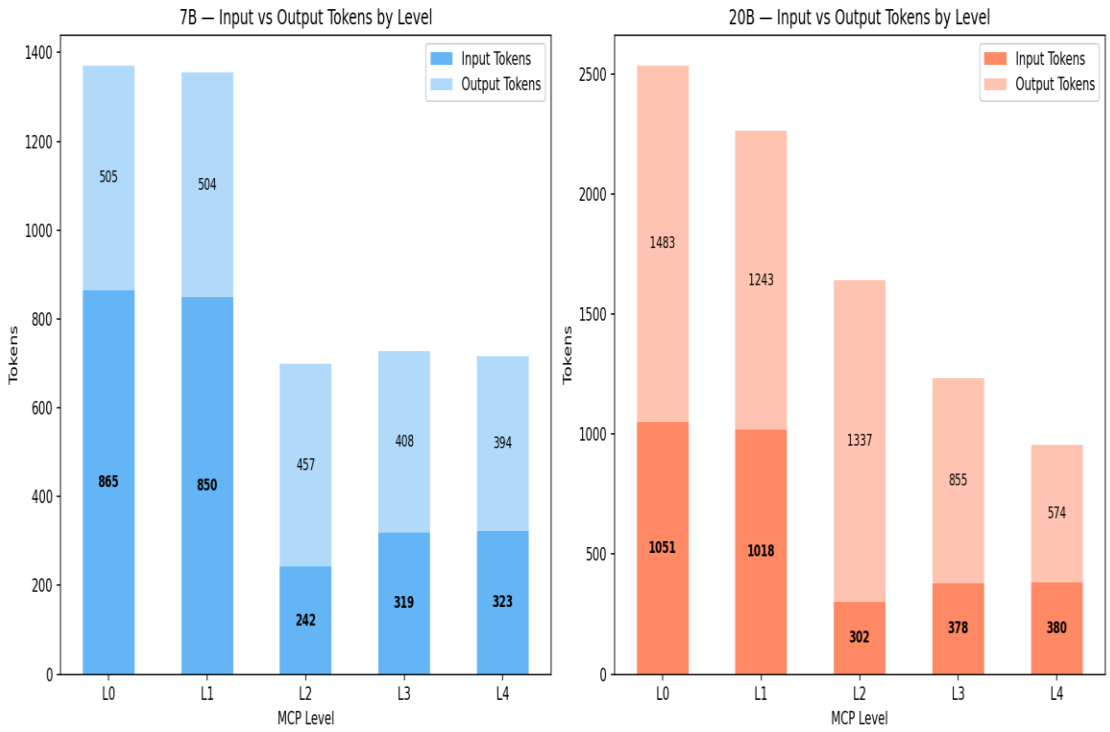

Figure 4. Input/Output Token Breakdown

4.2 Empty Response Analysis

Of the 21 empty responses from 20B, 17 were concentrated at L3/L4, particularly in code generation scenarios (S05 L4, S07 L3/L4). This suggests decoding instability when the 20B model attempts long sequence generation on commodity CPU hardware. Code output is more vulnerable than natural language due to higher inter-token dependencies. Figure 5 shows the distribution of empty responses by level and scenario.

This result has practical implications for model size selection in commodity environments: **bigger is not always better; there exists an environment-appropriate model size.**

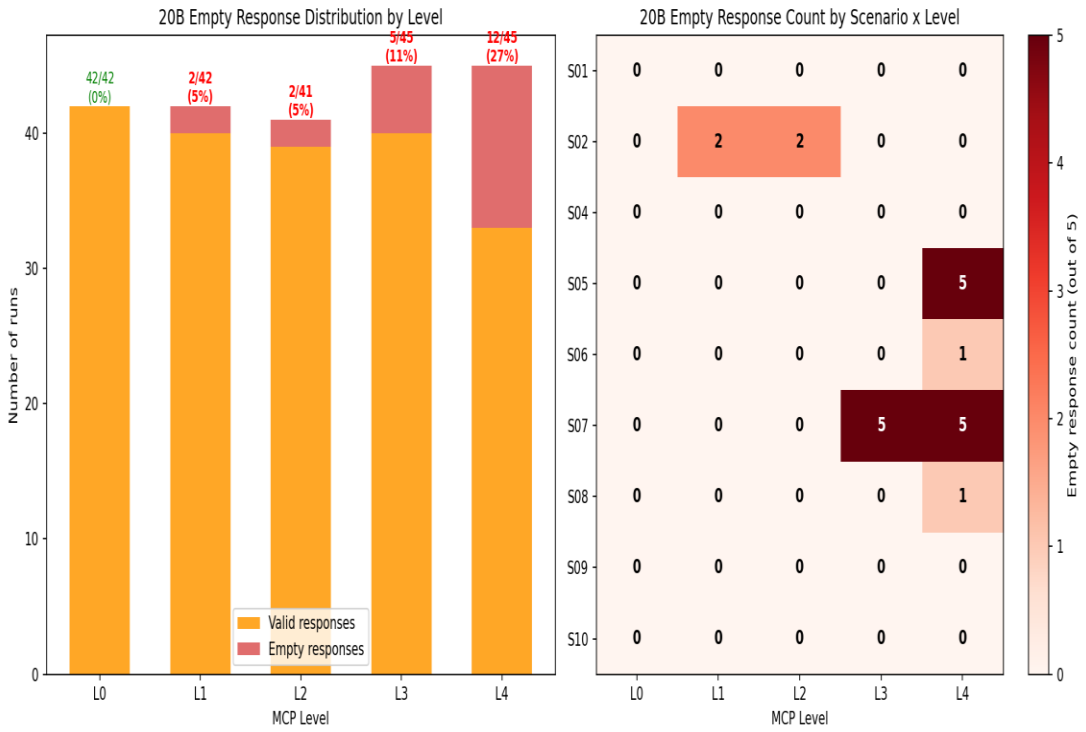

Figure 5. Empty Response Distribution Analysis

4.3 Clean Comparison: 7B vs. 20B

Core Comparison: 7B+L3 vs. 20B+L0

Table 8. 7B+L3 vs. 20B+L0 — Key Comparison

Metric	7B+L3	20B+L0	Difference
Total Tokens	728	2,534	3.5× cheaper
Accuracy	0.936	0.792	14.4%p higher
Input Tokens	319	1,051	3.3× fewer
Output Tokens	408	1,483	3.6× fewer

This comparison encapsulates one of our core messages: **a small model (7B) with well-designed MCP structure (L3) is 3.5× cheaper and 14.4%p more accurate than a large model (20B) operating without MCP.** However, this comparison has two caveats: (1) 20B exhibited 9.8% empty responses indicating hardware-level instability, and (2) the comparison is asymmetric—L3 provides structured context while L0 provides raw data. A fairer comparison would be 7B+L3 vs. 20B+L3 (clean accuracy: 0.936 vs. 0.996), where 20B's advantage partially re-emerges.

At L0–L2, model size dominates and 20B provides better answers from its inherent knowledge. However, at L3 and above, MCP structure becomes dominant—both models achieve 0.93+ accuracy, and the importance of model size diminishes sharply. In sufficiently structured MCP environments, MCP design matters more than model size. Figures 6 (raw vs. clean comparison) and 7 (clean scatter plot) visualize

this transition point.

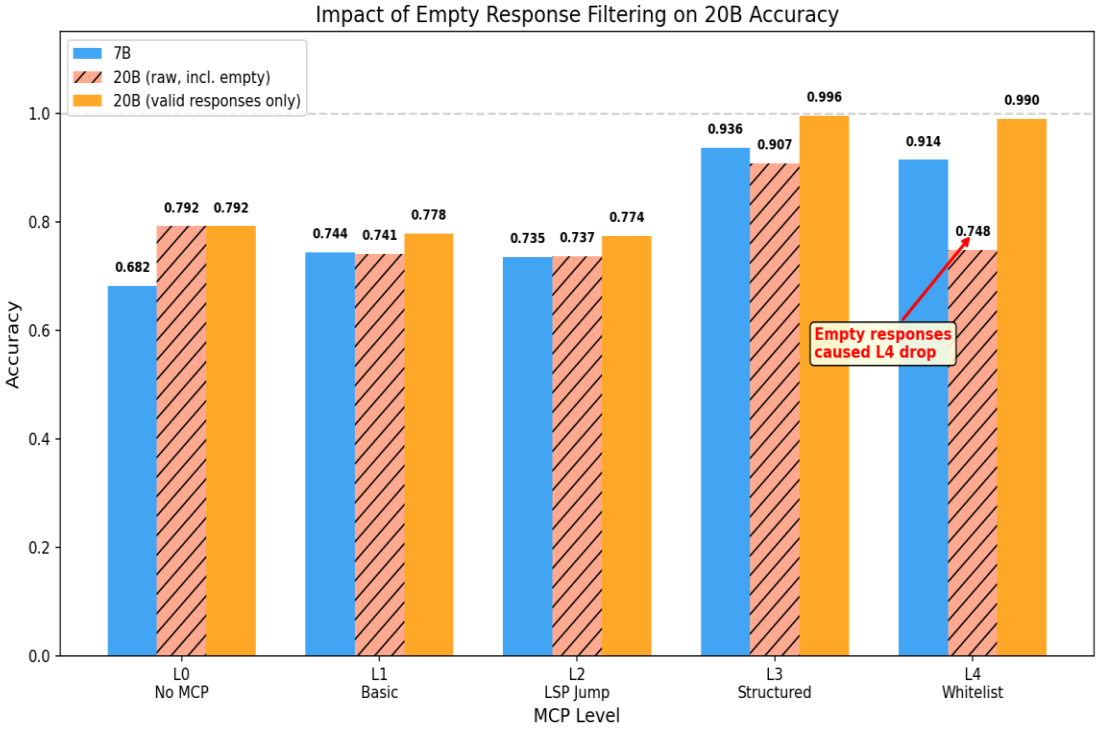

Figure 6. Raw vs. Clean Accuracy Comparison

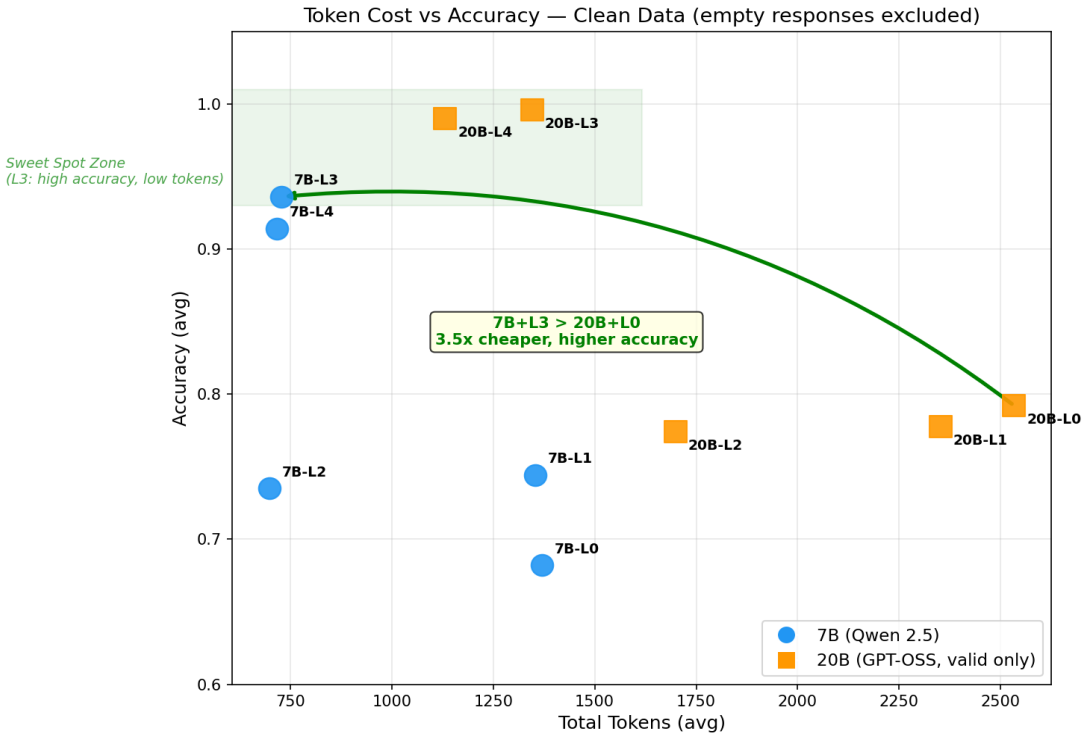

Figure 7. Clean Token-Accuracy Scatter

4.4 Scenario Analysis: Difficulty-Dependent MCP Contribution

Figures 8 (7B heatmap) and 9 (20B heatmap) visualize per-scenario accuracy across MCP Levels. Easy scenarios (e.g., S01) show high accuracy across all levels, while difficult scenarios (S08, S09) show dramatic improvement starting at L3.

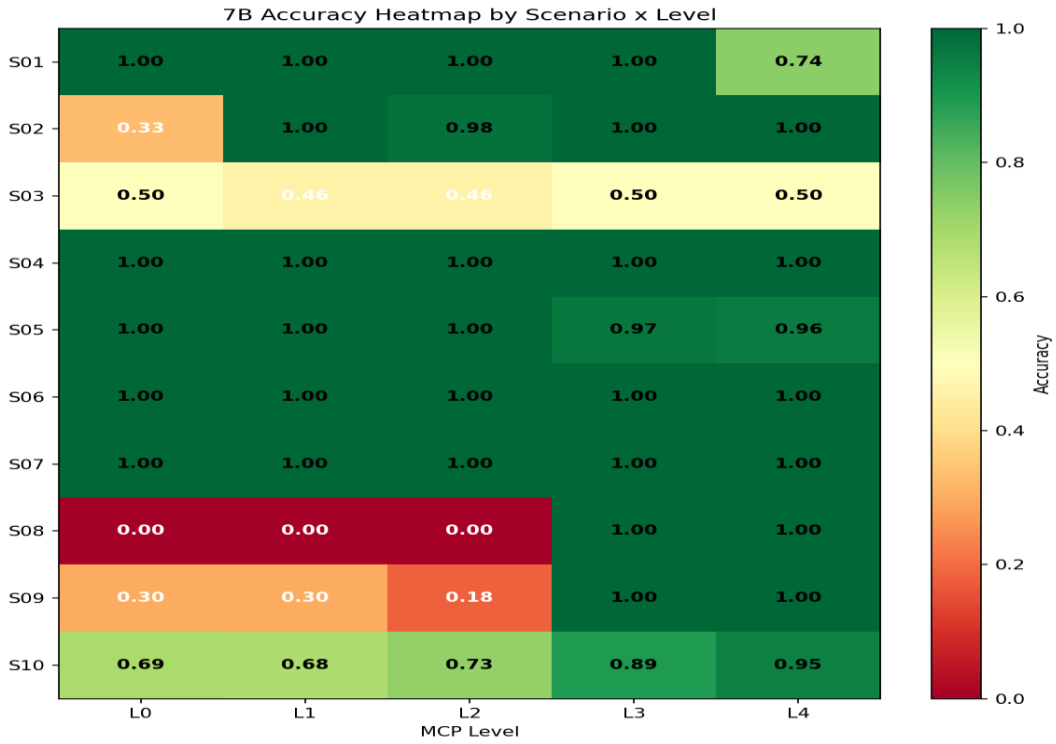

Figure 8. 7B Model Accuracy Heatmap by Scenario

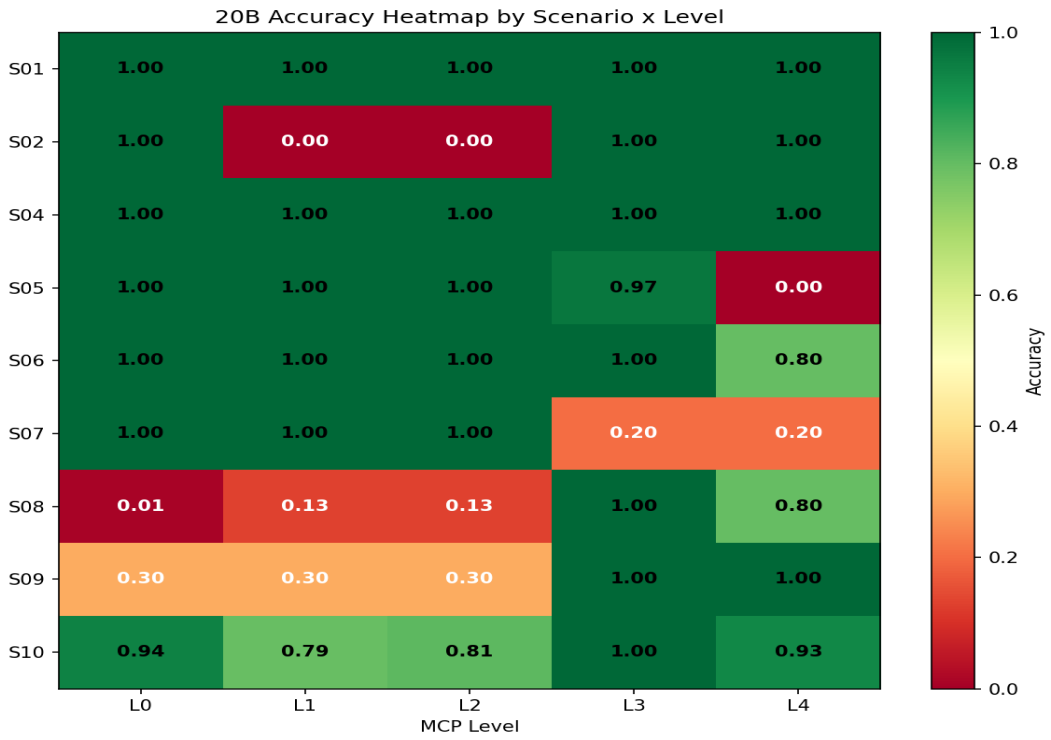

Figure 9. 20B Model Accuracy Heatmap by Scenario

4.4.1 Easy Scenarios (7B L0 ≥ 0.96)

S01 (DB Diagnosis), S04 (Code Navigation), S05 (Structure), S06 (Bug Fix), and S07 (Refactoring) achieved high accuracy even at L0. MCP's primary effect here is token savings. Notably, S01 saw 7B L4 drop to 0.74, suggesting that excessive restriction can cause adverse effects even in easy scenarios.

4.4.2 Difficult Scenarios (MCP essential)

Table 9. Difficult Scenarios — MCP is Essential

Scenario	7B L0	7B L3	20B L0 (clean)	20B L3 (clean)
S08 (Auto-Remediation)	0.00	1.00	0.01	1.00
S09 (AI-Assisted Judgment)	0.30	1.00	0.30	1.00

These scenarios constitute the **core evidence of this study.** At L0, agents cannot even identify which actions are available (e.g., "cancel_query"). At L3, when "recommended action: cancel_query" + "dangerous actions: restart, kill_backend" are presented, all 5 repeated runs select the correct action. **The 0→1.00 jump in S08** demonstrates that MCP's "candidate presentation + rationale provision" is decisive.

4.4.3 Scenario Pattern Summary

Easy (S01, S04, S05, S06, S07): L0 ≈ 1.0 → MCP = primarily token savings. Medium (S02, S10): L0: 0.33–0.69 → MCP = accuracy + token savings. Difficult (S08, S09): L0: 0.00–0.30 → MCP = essential (under our conditions, practically impossible without it). This pattern provides evidence for RQ1: MCP

contribution differs by layer. In upper-layer tasks, MCP is an efficiency tool; in lower-layer tasks, MCP is essential infrastructure.

4.5 Safety Analysis

In S08 and S09, L4 applies action whitelists (e.g., S08 allows cancel_query, kill_query but blocks restart, kill_backend, vacuum). Both L3 and L4 maintain accuracy of 1.00, meaning the whitelist provides additional safety without impacting accuracy in these scenarios.

Synthesizing the safety-accuracy relationship: **MCP simultaneously improves efficiency, accuracy, and safety, but the optimal point may differ along each axis.** The accuracy optimum is L3, the safety optimum is L4. In production environments where safety takes priority, L4 may be appropriate despite slight accuracy trade-offs.

Connecting to the real incidents mentioned in Section 1: the OpenClaw email deletion would have been blocked by L4's whitelist prohibiting "delete" actions; Replit's DB deletion would have been prevented by blocking drop/delete as forbidden actions; and Kiro's autonomous environment deletion could have been prevented by L4's environment-level restrictions.

4.6 Statistical Validation

Given the preliminary sample size (n=5 per condition), we apply nonparametric tests to avoid distributional assumptions. Five of ten scenarios showed ceiling effects (L0 accuracy already 1.0), so accuracy tests were restricted to the six non-ceiling scenarios (S02, S03, S05, S08, S09, S10).

Table 10. Statistical Tests (Wilcoxon Signed-Rank + Bootstrap 95% CI)

Comparison	n	Test	W	z	p-value	95% CI
Accuracy L0→L3 (non-ceiling)	30	Wilcoxon signed-rank	12	−3.83	0.0001***	[+0.287, +0.564]
Tokens L0→L3 (all scenarios)	50	Wilcoxon signed-rank	199	−4.23	<0.0001***	[+370, +964]
Accuracy L3→L4 (all scenarios)	50	Wilcoxon signed-rank	18	−0.53	0.594 (ns)	[−0.008, +0.064]

Key statistical findings: (1) The L0→L3 accuracy improvement is highly significant (p=0.0001, Wilcoxon signed-rank), with a 95% bootstrap CI of [+0.287, +0.564]. Combined with the previously reported Cohen's d=0.925, the effect is both statistically and practically significant. (2) Token reduction is equally significant (p<0.0001). (3) **Crucially, L3 vs. L4 shows no statistically significant difference (p=0.594),** formally confirming L3 as the sweet spot—L4's additional restrictions provide safety benefits without measurable accuracy gains.

4.7 L2.5 Ablation: Decomposing the L2→L3 Jump

The L2→L3 transition adds multiple features simultaneously: KB search results, candidate actions, call hierarchies, and diagnostic information. To isolate which component drives the accuracy jump

(0.735→0.936), we introduced an intermediate condition L2.5 defined as L2 + KB search results only (no recommended actions, no call hierarchy, no chained analysis). An additional 50 runs (10 scenarios × 5 repetitions) were executed for the 7B model.

Table 11. L2.5 Ablation Results (7B model, n=5 per cell)

Scenario	L2	L2.5 (L2+KB)	L3	L2→L2.5	L2.5→L3	Key Driver
S01 DB Diagnosis	1.00	0.93	1.00	−0.07	+0.07	Ceiling
S02 Error Analysis	0.98	0.65	1.00	−0.33	+0.35	Recommended actions
S03 Security Eval	0.46	0.20	0.50	−0.26	+0.30	Recommended actions
S04 Code Navigation	1.00	1.00	1.00	±0.00	±0.00	Ceiling
S05 Project Structure	1.00	0.62	0.97	−0.38	+0.35	Call hierarchy
S06 Bug Fix	1.00	0.00	1.00	−1.00	+1.00	Call hierarchy + diagnostics
S07 Refactoring	1.00	0.20	1.00	−0.80	+0.80	Call hierarchy
S08 Auto-Remediation	0.00	1.00	1.00	+1.00	±0.00	★ KB is decisive
S09 AI Judgment	0.18	0.30	1.00	+0.12	+0.70	Recommended actions
S10 Cascade Analysis	0.73	0.61	0.89	−0.12	+0.28	Chain analysis structure
Overall Mean	0.735	0.551	0.936	−0.184	+0.385	

Counterintuitive finding: Adding KB search alone (L2.5) **actually decreased overall accuracy from 0.735 to 0.551.** This is a significant negative result. In 6 of 10 scenarios, KB content introduced noise that degraded performance. The sole exception is S08 (Auto-Remediation), where the KB entry "cancel_query recommended for queries exceeding 600s" enabled a 0→1.00 jump even without explicit candidate actions.

This ablation reveals three distinct patterns:

(1) KB-driven tasks (S08): Domain-specific knowledge (e.g., "cancel_query for long-running queries") is the decisive factor. The model lacks this operational knowledge and cannot derive correct actions from metrics alone.

(2) Structure-driven tasks (S05, S06, S07): Call hierarchies and diagnostic information are decisive. These scenarios involve code comprehension where irrelevant KB content actively harms performance by distracting the model from the code context.

(3) Recommendation-driven tasks (S02, S09, S10): Candidate action presentation and structured analysis (not raw KB) drive the improvement. KB provides partial benefit only when paired with explicit recommendations.

Implication: L3's effectiveness is not attributable to any single component but to the **synergistic combination of relevant context types matched to task requirements.** Indiscriminate addition of context (e.g., KB to code tasks) is counterproductive. This supports a design principle of task-aware

context curation rather than maximal context provision.

4.7.1 L3 Component Decomposition: Recommended Actions vs. Call Hierarchy

The L2.5 ablation in Section 4.7 established that KB search is not the primary driver of L3's performance gains. However, the individual contributions of the remaining two components—recommended actions and call hierarchy—were not isolated. To address this, we define two additional conditions and execute 100 additional runs (2 conditions × 10 scenarios × 5 repetitions) on the 7B model.

L2.5a (L2 + recommended actions only): Adds recommended actions to L2 baseline. KB search and call hierarchy are excluded.

L2.5b (L2 + call hierarchy only): Adds call hierarchy information to L2 baseline. KB search and recommended actions are excluded.

Table 12. L3 Component Decomposition Ablation Results (7B model)

Condition	Components	Accuracy	vs L3	Notable Scenarios
L3 (full)	KB+Actions+Hierarchy	0.735	—	baseline
L2.5 (KB only)	KB only	0.551	−25%	S08: 0→1.00 (KB-driven)
L2.5a (actions)	Actions only	0.503	−32%	S08: 1.00 (direct guidance)
L2.5b (hierarchy)	Hierarchy only	0.366	−50%	S08: 0.00 (structure alone fails)

Synergy effect. No single component can substitute for the full L3 (0.735). Both L2.5a (actions only, 0.503) and L2.5b (hierarchy only, 0.366) show substantial performance degradation compared to L3, confirming that L3's effectiveness arises from the combinatorial synergy of its components rather than any individual contribution.

Actions > Hierarchy. Recommended actions (L2.5a=0.503) contribute more than call hierarchy (L2.5b=0.366). The most dramatic divergence occurs in S08 (threshold response): L2.5a=1.000 vs. L2.5b=0.000. The recommended actions include explicit operational directives (e.g., "cancel_query when exceeding 600s"), which provide directly actionable guidance that structural information alone cannot replicate.

No single component suffices. S06 and S09 score 0.000 across all three isolated conditions (L2.5, L2.5a, L2.5b), demonstrating that these tasks require all components working in concert. This evidences that L3's design philosophy—providing multi-layered, task-appropriate context—produces a qualitative transition rather than an additive improvement.

4.8 Pylon-7 Layer ↔ Scenario Mapping

Table 1 defined 7 Pylon-7 layers theoretically. We now map each experimental scenario to its corresponding Pylon-7 layer based on task characteristics, providing the empirical grounding that connects the theoretical framework to experimental results.

Table 12. Pylon-7 Layer ↔ Scenario Mapping

Scenario	Pylon-7 Layer	Optimal MCP	Role Classification
S01 DB Diagnosis	L3 Resource	L2+	Read-only resource access → Essential infra
S02 Error Analysis	L6 Analysis	L3	Input analysis + KB → Efficiency tool
S03 Security Eval	L6 Analysis	L3	External knowledge judgment → Efficiency tool
S04 Code Navigation	L3 Resource	L2	Symbol-level access → Essential infra
S05 Project Structure	L3 Resource	L3	Resource + call relations → Essential infra
S06 Bug Fix	L2 Mutation	L2	Code mutation → Essential infra
S07 Refactoring	L2 Mutation	L3	Code mutation + structure → Essential infra
S08 Auto-Remediation	L4 Service	L2.5/L3	Service invocation → Mid-layer
S09 AI Judgment	L5 Routing	L3	Routing decision + correlation → Efficiency tool
S10 Cascade Analysis	L5 Routing	L3	Multi-event routing → Efficiency tool

This mapping validates the claim in Section 5.1: MCP serves as an efficiency tool for upper-layer tasks (L5–L6: analysis, routing) and as essential infrastructure for lower-layer tasks (L2–L3: mutation, resource access). The L2.5 ablation further supports this—lower-layer tasks (S05–S07) depend on structural context (call hierarchies), while upper-layer tasks (S08–S10) depend on domain knowledge and recommendations.

5. Discussion

5.1 Research Question Answers

RQ1 (Benefit of 7-layer decomposition): Results show MCP contribution differs by layer—efficiency tool for upper layers, essential infrastructure for lower layers. This aligns with the Descent Cost Principle and cannot be explained by single-layer models.

RQ2 (MCP depth → cost/accuracy): L0→L3 yielded 47% token reduction + 37% accuracy improvement for 7B (n=250), and 51% token reduction + 15% accuracy improvement for 20B. Bidirectional improvement occurs regardless of model size.

RQ3 (Sweet spot): L3 is the sweet spot, confirmed statistically (L3 vs. L4: p=0.594, non-significant). The L2.5 ablation (Section 4.7) reveals that L3's effectiveness stems not from KB search alone but from the synergistic combination of structured recommendations and call hierarchies matched to task type. **An optimal constraint level exists, and excessive constraint is counterproductive.**

RQ4 (3-stage pipeline): Directly validated in S08 (0→1.00) and S09 (0.30→1.00). Deterministic rules filter clear cases at Stage A, enabling AI to judge "already-narrowed candidates"—superior in both cost and accuracy to single AI calls.

RQ5 (Small+MCP vs. Large): 7B+L3 (728 tokens, 0.936) vs. 20B+L0 (2,534 tokens, 0.792). 3.5× cheaper and 14.4%p more accurate. Above L3, model size influence diminishes sharply as both models achieve 0.93+.

Effect Size: For the 7B model, the L0→L3 accuracy improvement yields Cohen's d = 0.925 (large effect), confirming that this difference is practically significant. Token reduction shows Cohen's d =

−0.818 (large effect). For the 20B model, token reduction is even larger at d = −1.630 (large effect).

5.2 The "Constraint Improves Performance" Thesis

The most counterintuitive finding is that structural constraint through MCP simultaneously reduces tokens and improves accuracy. At L0, the agent must independently filter relevant information from the entirety of code, system state, and possible actions. This wide "selection scope" consumes more tokens (exploration cost) and increases the probability of wrong selections (accuracy degradation).

At L3, MCP (1) extracts only relevant symbols to reduce input, (2) provides similar cases through KB search, and (3) narrows the selection range by presenting candidate actions. This is far more effective than "opening everything and leaving it to the AI." The L2.5 ablation (Section 4.7) reveals nuance: component (2) alone is insufficient and can be harmful; it is the combination of structured recommendations matched to task type that produces the constraint benefit.

Therefore, **optimal MCP design is "sufficient structuring + appropriate constraint,"** which is precisely why L3 is the sweet spot.

5.3 Model Size vs. MCP Structure

Our results challenge the conventional wisdom that "bigger models are always better." At L0–L2, model size dominates, but at L3 and above, MCP structure takes over. In commodity environments, the 20B model exhibits empty responses (9.8%), long inference times, and excessive token output, while 7B achieves 250/250 normal responses with consistent performance. An appropriately-sized model + well-designed MCP is practically superior to an oversized model for the environment.

5.4 Limitations and Future Work

Statistical limitations: With 5 repetitions per scenario (n=5), nonparametric tests have limited statistical power. Additionally, 5 of 10 scenarios (S01, S04, S05, S06, S07) showed ceiling effects where both L0 and L3 achieved 1.0 accuracy, rendering paired comparisons meaningless. Effect sizes (Cohen's d = 0.925, large effect) confirm practical significance, but future work should increase to n≥30 repetitions and diversify difficulty to resolve ceiling effects.

Internal validity: Keyword matching proved inadequate for open-ended scenarios (S03). The 21 empty responses from 20B require further investigation. Evaluation functions differ across scenarios, limiting cross-scenario comparisons.

External validity: This study deliberately targets commodity hardware (consumer-grade CPU, no GPU) with small models (Qwen2.5 7B/20B). This is an intentional design choice, not a limitation: since high-end environments can achieve sufficient performance without MCP, the protocol's value must be demonstrated precisely where it is most needed. Generalization to other model families (LLaMA, Mistral, Gemma) and domains (code generation, data analysis) remains for future work, but this study's core contribution—empirical proof of MCP efficacy in resource-constrained environments—is enabled by this design.

Construct validity: MCP Levels L0–L4 are discrete 5-step levels; the actual optimum may lie on a continuous spectrum. The L2→L3 gap, which involved compound feature additions, has been partially addressed by the L2.5 ablation study (Section 4.7), revealing that structured recommendations and call hierarchies—not KB search alone—drive the accuracy improvement. A follow-up L3 component decomposition experiment (Section 4.7.1) confirmed that neither recommended actions alone (0.503) nor call hierarchy alone (0.366) can match the full L3 (0.735), establishing that the synergy between components is the key driver of L3 performance.

6. Conclusion

6.1 Key Findings

This study proposes Pylon-7, a 7-layer reference model for AI agent workflows, and presents exploratory evidence of MCP layering's simultaneous effects on efficiency, accuracy, and safety through 615 experimental runs (465 main + 150 ablation) in a commodity hardware environment. The six key findings are:

1. **MCP layering simultaneously improves tokens and accuracy.** L0→L3: 47% token reduction + 37% accuracy improvement (Wilcoxon p=0.0001, 95% CI [+0.287, +0.564]; Cohen's d=0.925). Not a trade-off but a win-win.
2. **L3 is the sweet spot.** L3 vs. L4 difference is statistically non-significant (p=0.594). L4 provides safety benefits without measurable accuracy gains.
3. **Small+MCP > Large alone.** 7B+L3 (728 tokens, 0.936) vs. 20B+L0 (2,534 tokens, 0.792). 3.5× cheaper, 14.4%p more accurate. Note: 20B showed 9.8% empty responses, so this comparison should be interpreted with appropriate caution.
4. **MCP is decisive for difficult tasks.** S08: 0→1.00, S09: 0.30→1.00. Under our experimental conditions, tasks unachievable without MCP become perfectly executable with it.
5. **L2→L3 improvement is driven by structured recommendations, not KB search.** L2.5 ablation (L2+KB only) decreased accuracy from 0.735 to 0.551. Further component decomposition showed that recommended actions alone (0.503) and call hierarchy alone (0.366) both fall short of L3, confirming that the synergy among components—not any single element—drives L3 effectiveness.
6. **Context components are task-specific.** S08 benefits from KB (0→1.00), while S06/S07 are harmed by it (1.00→0.00). Effective MCP design must match context type to task characteristics.

6.2 Practical Guidelines

1. **Simple tasks don't need MCP.** For numeric extraction and code navigation, MCP's primary value is token savings.
2. **Apply L3-level MCP for complex judgment tasks.** Provide structured context + candidate actions rather than full code.

3. **Apply L4 in safety-critical production.** Accept slight accuracy trade-offs for dangerous action prevention.
4. **Improve MCP before scaling model size.** 7B+L3 outperforms 20B+L0—well-designed MCP beats bigger models.
5. **Actively leverage MCP's "restriction" capability.** MCP's value lies not only in opening tools but also in closing them. Appropriate constraint simultaneously achieves efficiency, accuracy, and safety.
6. **Curate context by task type, not by volume.** For operational policy tasks (S08-type), KB entries are effective; for code tasks (S06/S07-type), prioritize call hierarchies and omit KB. Indiscriminate context addition degrades performance (L2.5 ablation, Section 4.7).

6.3 Closing Remarks

MCP's core value lies not only in "extension" but in **"structural constraint."** Structuring and constraining tools to match task context is more accurate, efficient, and safer than granting unrestricted access. This study presents exploratory evidence supporting this counterintuitive thesis in a commodity hardware environment. Generalization requires further large-scale experiments across diverse models and domains, which we leave to future work.

References

[1] Anthropic. “Model Context Protocol (MCP).” 2024. https://modelcontextprotocol.io

[2] K. Huang. “The AI Agentic Ecosystem: A 7-Layer Reference Architecture.” Medium, Dec. 2024; also in Springer “Agentic AI: Theories and Practices,” Ch. 2, 2025.

[3] W. Hou et al. “Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions.” arXiv:2503.23278, 2025.

[4] M. Hasan et al. “An Empirical Study of Model Context Protocol Servers: Code Quality and Security Analysis of 1,899 MCP Servers.” arXiv:2506.13538, 2025.

[5] A. Lodha. “TokenOps: A Compiler-Style Middleware for LLM Token Cost Optimization.” Preprint, 2025.

[6] Y. Wang et al. “Efficient Agents: Analyzing Module-Level Cost Contributions in AI Agent Systems.” arXiv:2508.02694, 2025.

[7] ISO/IEC 7498-1:1994. “Information technology — Open Systems Interconnection — Basic Reference Model.”

[8] S. Yao et al. “ReAct: Synergizing Reasoning and Acting in Language Models.” In Proc. ICLR, 2023. arXiv:2210.03629.

[9] T. Schick et al. “Toolformer: Language Models Can Teach Themselves to Use Tools.” In Proc. NeurIPS, 2023. arXiv:2302.04761.

[10] S. G. Patil et al. “Gorilla: Large Language Model Connected with Massive APIs.” In Proc. NeurIPS, 2024. arXiv:2305.15334.

[11] Y. Qin et al. “ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs.” In Proc. ICLR (Spotlight), 2024. arXiv:2307.16789.

[12] C. Qu et al. “Tool Learning with Large Language Models: A Survey.” Frontiers of Computer Science, 2024. arXiv:2405.17935.